Threat Modelling - STRIDE

Data Flow Diagram in a separate file called *Data-Flow-Diagram.png*.

Arrow/data Flow	STRIDE Threat Description	Priority	Existing Protection	What To Look Out For
Credentials -> Sign Up and Log in	Spoofing: Brute-force attempts, rainbow table attack, or dictionary attack on the */api/login*. If successful, they may end up authenticating as a legitimate user. DoS: attacker may flood on */api/create-user* with a lot of requests, which may be computationally demanding due to the hashing of the password.	High	Passwords are hashed and verified during login. Usernames are allowed to only contain letters, numbers, "-, "_"	A high number of traffic and 401 responses coming from a single IP (unless DDoS).
File payload -> Upload file	Tampering: The attacker uploads a malicious file with some runnable content while looking like a legitimate document. Could disrupt the application working or obtain a flag.	Critical	Uploading file requires a valid bearer token. Now, it has been changed that the file also needs to be a PDF format.	Responses 400 with *Only PDF files are allowed.*
File Request -> Handle User Files	Tampering: Attacker can attempt to change file paths in order to access different parts of the system and modify other files (Like Flag 1) Information Disclosure: Authenticated user attempts to access a file belonging to other users (attempt to get Flag 3 from Mr_Important).	Critical	While accessing a file, it is checked against the owner ID (also checked for the Assurance part of the course), and file paths are checked against a method to ensure that the path escape isn't possible.	High number of 404 Not Found when requesting files from other users, and 500 errors when a path is escaped.

PGP Messages -> Conduct RMAP Handshake	Spoofing: A peer client puts together a message to impersonate another group (Message 1). They could then get access to all of their watermarked files. Tampering: Attacker takes over Message 2 and replays it or reuses the session token. It could lead to rewriting all the files or messing up the versions of the files.	High	Checking the user against public keys found in *clients* and decrypting with its own private key. Replayed links are checked in the *Versions* and rejected with 409 *Conflict*.	Logs showing 400 Bad Request (*Unknown Identity*), and spikes of 409 errors.
Secret Link -> get File with Secret Link	Repudiation: Recipient downloads the linked document and leaks it. It could be discovered who has leaked it but not when and from which IP address due to no access logs. Information Disclosure: Adversary uses special tools to brute-force to get a secret link. If successful, they can download other group's files without authentication.	Medium	Links are created with a random hex string, which should make it too difficult to simply guess the link. Queries to the database also require parameters.	High number of 404 *Not Found.*
Watermark request -> Apply / Read Watermark	Information Disclosure: Attacker send a request to read a watermark from the document they do not own. If successful, they can see the embedded secrets. DoS: Adversary uploads a malicious PDF that triggers some script while running the watermarking, which ends up overwhelming the server.	Medium	Requests require bearer token and owner ID before accessing the files. There are also checks if watermarking is applicable first.	404 Not Found errors when trying to open unowned documents, or longer response time while DoS is being conducted.
Modify File -> D Storage Filesystem	Tampering: The attacker may try to escape the designated path in order to get access to the internal resources - e.g., source code, or Flag 2.	Critical	Docker ensures separation of the */app/* directory, making the root inaccessible.	Logs with mismatched SHA256 records in Documents or denied permission.

A collection of changes made on the project

1. Observation points to detect attacks.
    1. Configured Gunicorn to log all access and error output --*access-logfile, --error-logfile* and --*capture-output.* Keeps track of client IP addresses, requested endpoints, HTTP status codes to enable analysis.
    2. Added Secret Key checks when the application is run. The application now starts only if the secret key is present, otherwise, an error is logged and the access is refused.
2. Recover from attacks and prevent future ones.
    1. Deleted *load_plugin* endpoint in order to prevent uploading and triggering pickle files, where they could have hidden malicious code snippets before, which allowed for reading our flag. STRIDE: tampering, Elevation of Privilege.
    2. Rewriting the watermarking tool to disable shell commands from being injected. It was done by modifying the method, so that it only uses Python without opening a shell. STRIDE: Tampering, Elevation of Privilege.
    3. Implemented an identity check for the *delete-file* endpoint, so that only the user who owns a certain file can delete it, and it is impossible otherwise. STRIDE: Tampering, Information Disclosure.
    4. Added extra requirements for usernames - now, only letters, numbers, hyphens, and underscores are allowed. It is done so to prevent path injection inside the username input. If attempted, 400 *Bad Request* is returned. STRIDE: Tampering.
    5. Before it was possible to create a few users with the same usernames. Now, the database inquiry required every username to be unique, and rejects attempts to use the same username again. STRIDE: Spoofing.
    6. In order to prevent Message 2 in RMAP Handshake to be replayed, the server now checks *Versions* to see if certain link has already been created and returns *409 Conflict* if it find it in the table, without any processing of the file. STRIDE: Spoofing, Tampering.
    7. There was an attempt of an attack on our Database port (3306) and phpMyAdmin. In order to prevent that from being possible, the database has been put on an internal network, so it can only be accessed from the VM, and not from the entire network like it was before. STRIDE: Information Disclosure, Tampering.

A collection of relevant logs documenting attacks detected (successful or unsuccessful)

Unfortunately, we do not have access to all the logs of incidents as they would get removed every time we recomposed our Docker container. However, our team kept a journal of different incidents, describing what happened and our suspicion on how it could have been done.

Attack 1
15-21 September

Adversary uploaded a few files with *.pkl* format on *load_plugin*, which then executed code that allowed for capturing our flag. The file *x.pkl* was submitted repeatedly. As remediation, the *load_plugin* endpoint was removed.

Attack 2
13 October

Log: 10.100.11.11 - - [13/Oct/2026:07:05:26 +0000] "GET /api/get-version/%2e%2e%2fMr_Important%2f20261004T230601806573Z__flag_3.pdf HTTP/1.1" 404 207 "-" “curl/8.14.1"

Attack 3
5-6 October

The attackers scanned our ports with nmap, targeting ports 3306, 8080, and 5000, requesting /*phpmyadmin*/, /*pma*/, and /*.env*. The attacks failed (404. In response, 14th of October, the database was cut off from the external network and allowed to access only from our VM, which made it unreachable.

Attack 4
13 October

IP address of 10.100.11.37 uploaded, watermarked, read and deleted a file, which was repeated. It looked like an automated checker, but was still noted.
IP address of 10.100.11.15 sent around 1776 requests, which were analysed but inconclusively. There were requests to different endpoints without bearer tokens, which seemed like an attempt to find overlooked paths, or other vulnerabilities present. However, majority of them were met with status codes within the 400 family.